

ConcurrentStack<T> Collection Class in C# with Examples

In this article, I am going to discuss the **ConcurrentStack<T> Collection Class in C#** with Examples. Please read our previous article where we discussed [ConcurrentQueue Collection Class in C#](#) with Examples. At the end of this article, you will understand the following pointers.

1. [What is ConcurrentStack<T> Class in C#?](#)
2. [Why do we need ConcurrentStack<T> Collection Class in C#?](#)
3. [Generic Stack Example with Single Thread in C#](#)
4. [Generic Stack Example with Multi-Thread in C#](#)
5. [Generic Stack with Locking Mechanism in C#](#)
6. [ConcurrentStack<T> Collection Class with Multiple Threads in C#](#)
7. [How to Create a ConcurrentStack<T> Collection in C#?](#)
8. [How to Add Elements into a ConcurrentStack<T> Collection in C#?](#)
9. [How to Access a ConcurrentStack Collection in C#?](#)
10. [How to Remove Elements from ConcurrentStack<T> Collection in C#?](#)
11. [How to Get the Top Element from the ConcurrentStack in C#?](#)
12. [How to Copy a ConcurrentStack Collection to an Existing Array in C#?](#)
13. [ConcurrentStack<T> Collection Class with Complex Types in C#](#)
14. [Difference between Stack<T> and ConcurrentStack<T> in C#](#)

What is ConcurrentStack<T> Class in C#?

The **ConcurrentStack<T>** is a Thread-Safe Collection Class in C#. It was introduced as part of .NET Framework 4.0 and it belongs to `System.Collections.Concurrent` namespace. It provides a thread-safe Last-In-First-Out (LIFO) data structure. That means we need to go for **ConcurrentStack<T> Collection** when we need Last in First Out (LIFO) access to the collection elements in a multi-threaded environment with thread safety.

The working of the **ConcurrentStack<T>** is very much similar to the working of the **Generic Stack<T> Collection Class in C#**. The only difference between them is that the **Generic Stack<T> Collection** is not thread-safe whereas the **ConcurrentStack<T>** is thread-safe. So, we can use the **Generic Stack** class instead of **ConcurrentStack** with multiple threads, but in that case, as a developer, we need to use locks explicitly to provide thread safety which is always time-consuming and error-prone. So, the ideal choice is to use **ConcurrentStack<T>** instead of **Generic Stack<T>** in a multi-threaded environment, and with **ConcurrentStack<T>**, as a developer, we don't require to implement any locking mechanism explicitly. The **ConcurrentStack<T>** collection class handles thread safety internally.

Why do we need ConcurrentStack<T> Collection Class in C#?

Let us understand why we need **ConcurrentStack** collection class in C# with some examples. So, what we will do here is, first, we will see examples using **Generic Stack**, then we will see the thread-safety problem with **Generic Stack** and how we can resolve the thread-safety problem by implementing the locking mechanism explicitly, and finally, we will see how to use **ConcurrentStack** collection class provided by `System.Collections.Concurrent` namespace.

Generic Stack Example with Single Thread in C#:

In the following example, we created a generic stack called MobileOrders to store order information for mobile. Further, if you notice in the below code, the GetOrders method is called from the TestStack method in a regular synchronous way. And from the main method, we simply call the TestStack method.

```
using System;
using System.Collections.Generic;
using System.Threading;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            TestStack();
            Console.ReadKey();
        }
        public static void TestStack()
        {
            Stack<string> MobileOrders = new Stack<string>();
            GetOrders("Pranaya", MobileOrders);
            GetOrders("Anurag", MobileOrders);
            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order Placed: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, Stack<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {
                Thread.Sleep(100);
                string order = string.Format($"{custName} Needs {i + 3} Mobiles");
                MobileOrders.Push(order);
            }
        }
    }
}
```

Output:

```
Order Placed: Anurag Needs 5 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Anurag Needs 3 Mobiles
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Pranaya Needs 3 Mobiles
```

As the GetOrders method is called in a synchronous way, the output is also printed similarly i.e. first Pranaya and then Anurag which is what you can see in the above output.

Generic Stack Example with Multi-Thread in C#:

Now, let's modify the previous example to make it asynchronous. For that, we have used Task that will call the GetOrders method by using two different threads. And we have done these changes inside the TestStack method as shown in the below code.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            TestStack();
            Console.ReadKey();
        }
        public static void TestStack()
        {
            Stack<string> MobileOrders = new Stack<string>();

            Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
            Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
            Task.WaitAll(t1, t2); //Wait till both the task completed

            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order Placed: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, Stack<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {
                Thread.Sleep(100);
                string order = string.Format($"{custName} Needs {i + 3} Mobile");
                MobileOrders.Push(order);
            }
        }
    }
}
```

Now, run the above code multiple times and each time you will get a different output. That means the output is not consistent as shown in the below image.

```
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Anurag Needs 3 Mobiles
Order Placed:
```

Why we are not getting the expected output?

This is because the Push method of Generic Stack<T> Collection Class is not designed to work with more than one thread parallelly i.e. the Push method is not thread safety. So, Multi-Threading with Generic Stack<T> is unpredictable. That means, sometimes It may work but if you try it several times, then will get unexpected results.

Generic Stack with Locking Mechanism in C#:

In the following example, we use the lock keyword to the statement i.e. the statement which adds the order to the stack i.e. the Push method.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentStackDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            TestStack();
            Console.ReadKey();
        }
        public static void TestStack()
        {
            Stack<string> MobileOrders = new Stack<string>();

            Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
            Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
            Task.WaitAll(t1, t2); //Wait till both the task completed

            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order Placed: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, Stack<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {

```

```

        Thread.Sleep(100);
        string order = string.Format($"{custName} Needs {i + 3} Mobiles");
        lock (lockObject)
        {
            MobileOrders.Push(order);
        }
    }
}

```

Now, run the above code and you will get the output as expected as shown in the below image.

```

Order Placed: Anurag Needs 5 Mobiles
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Pranaya Needs 3 Mobiles
Order Placed: Anurag Needs 3 Mobiles

```

That is fine. So, getting expected results after putting a lock on the Push method. But what about if the Push is called multiple times at multiple locations in our application, would you like to use the lock statement everywhere? If you do so, then it is a time-consuming process as well as error-prone as you might forget to use a lock somewhere. The solution is to use ConcurrentStack.

ConcurrentStack<T> Collection Class with Multiple Threads in C#:

The ConcurrentStack provides thread safety automatically in a multi-threaded environment. Let us rewrite the previous example using the ConcurrentStack collection class and see the output and then we will discuss ConcurrentStack collection class in detail. In the following example, we simply replace the Stack class with ConcurrentStack. And remove the statement used for explicitly locking. Please note the ConcurrentStack class belongs to System.Collections.Concurrent namespace, so include that namespace.

```

using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            TestStack();
            Console.ReadKey();
        }
        public static void TestStack()
        {
            ConcurrentStack<string> MobileOrders = new ConcurrentStack<string>();
            Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));

```

```

Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
Task.WaitAll(t1, t2); //Wait till both the task completed
foreach (var mobileOrder in MobileOrders)
{
    Console.WriteLine($"Order Placed: {mobileOrder}");
}
}
private static void GetOrders(string custName, ConcurrentStack<string>
MobileOrders)
{
    for (int i = 0; i < 3; i++)
    {
        Thread.Sleep(100);
        string order = string.Format($"{custName} Needs {i + 3} Mobiles");
        MobileOrders.Push(order);
    }
}
}
}

```

Output:

```

Order Placed: Anurag Needs 5 Mobiles
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Pranaya Needs 3 Mobiles
Order Placed: Anurag Needs 3 Mobiles

```

Now, I hope you understand the basic need for the ConcurrentStack collection class. Let us proceed and understand C# ConcurrentStack Collection Class in detail.

Methods, Properties, and Constructors of ConcurrentStack Class in C#:

Let us understand the different Methods, Properties, and Constructors of ConcurrentStack Collection Class in C#. If you right-click on the ConcurrentStack class and select go to definition, then you will see the following definition. The ConcurrentStack class belongs to System.Collections.Concurrent namespace and implements IProducerConsumerCollection<T>, IEnumerable<T>, IEnumerable, ICollection, IReadOnlyCollection<T> interfaces.

```

namespace System.Collections.Concurrent
{
    ...public class ConcurrentStack<T> : IProducerConsumerCollection<T>, IEnumerable<T>, IEnumerable, ICollection, IReadOnlyCollection<T>
    {
        ...public ConcurrentStack();
        ...public ConcurrentStack(IEnumerable<T> collection);

        ...public bool IsEmpty { get; }
        ...public int Count { get; }

        ...public void Clear();
        ...public void CopyTo(T[] array, int index);
        ...public IEnumerator<T> GetEnumerator();
        ...public void Push(T item);
        ...public void PushRange(T[] items);
        ...public void PushRange(T[] items, int startIndex, int count);
        ...public T[] ToArray();
        ...public bool TryPeek(out T result);
        ...public bool TryPop(out T result);
        ...public int TryPopRange(T[] items);
        ...public int TryPopRange(T[] items, int startIndex, int count);
    }
}

```

How to Create a ConcurrentStack<T> Collection in C#?

The ConcurrentStack<T> Collection Class in C# provides the following two constructors to create an instance of the ConcurrentStack<T> class.

1. **ConcurrentStack():** It is used to initialize a new instance of the ConcurrentStack class.
2. **ConcurrentStack(IEnumerable<T> collection):** It is used to initialize a new instance of the ConcurrentStack class that contains elements copied from the specified collection.

Let's see how to create an instance of ConcurrentStack using the ConcurrentStack() constructor:

Step1:

As the ConcurrentStack<T> class belongs to System.Collections.Concurrent namespace, so first, we need to include System.Collections.Concurrent namespace in our program is as follows:

```
using System.Collections.Concurrent;
```

Step2:

Next, we need to create an instance of the ConcurrentStack class using the ConcurrentStack() constructor as follows:

```
ConcurrentStack<type> ConcurrentStack _Name = new ConcurrentStack<type>();
```

Here, the type can be any built-in data type like int, double, string, etc., or any user-defined data type like Customer, Student, Employee, Product, etc.

How to Add Elements into a ConcurrentStack<T> Collection in C#?

If you want to add elements to a ConcurrentStack collection in C#, then you need to use the following methods of the ConcurrentStack<T> class.

1. **Push(T item):** This method is used to insert an object at the top of the ConcurrentStack.
2. **PushRange(T[] items):** This method is used to insert multiple objects at the top of the ConcurrentStack atomically.
3. **PushRange(T[] items, int startIndex, int count):** This method is used to insert multiple objects at the top of the ConcurrentStack atomically. Here, the parameter items specify the objects to push onto the ConcurrentStack. The parameter startIndex specifies the zero-based offset in items at which to begin

inserting elements onto the top of the ConcurrentStack. And, the parameter count specifies the number of elements to be inserted onto the top of the ConcurrentStack.

For example,

```
ConcurrentStack<string> concurrentStack = new ConcurrentStack<string>();
```

The above statement will create a ConcurrentStack of string types. So, here we can only add string values on the ConcurrentStack. If you try to add anything other than a string then you will get a compile-time error.

```
concurrentStack.Push("India");
```

```
concurrentStack.Push("USA");
```

```
concurrentStack.Push(100); //Compile-Time Error
```

Adding multiple elements using PushRange(T[] items) method.

Creating a string array: `string[] countriesArray = { "UK", "NZ" };`

Adding string array to ConcurrentStack using PushRange Method

```
concurrentStack.PushRange(countriesArray);
```

Note: We cannot add elements into a ConcurrentStack using Collection Initializer.

How to access a ConcurrentStack Collection in C#?

We can access all the elements of the ConcurrentStack collection in C# using a for each loop as follows.

```
foreach (var item in concurrentStack)
{
    Console.WriteLine(item);
}
```

Example to Understand How to Create a ConcurrentStack and Add Elements in C#:

For a better understanding of how to create a ConcurrentStack, how to add elements, and how to access all the elements from ConcurrentStack in C# using a for-each loop, please have a look at the following example which shows the above three things.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            //Creating concurrentStack to store string values
            ConcurrentStack<string> concurrentStack = new ConcurrentStack<string>();
            //Adding Element using Push Method of ConcurrentStack Class
            //Only one element at a time
            concurrentStack.Push("India");
            concurrentStack.Push("USA");
            //concurrentStack.Push(100); //Compile-Time Error
            Console.WriteLine("ConcurrentStack Elements after Push Method");
            foreach (var item in concurrentStack)
            {
                Console.WriteLine(item);
            }
        }
    }
}
```

```

    }

    //Creating a string array
    string[] countriesArray = { "UK", "NZ" };

    //Adding Elements to ConcurrentStack using PushRange Method
    //Adding collection at a time
    concurrentStack.PushRange(countriesArray);

    Console.WriteLine("\nConcurrentStack Elements after PushRange Method");
    foreach (var item in concurrentStack)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
}
}

```

Output:

```

ConcurrentStack Elements after Push Method
USA
India

ConcurrentStack Elements after PushRange Method
NZ
UK
USA
India

```

How to Remove Elements from ConcurrentStack<T> Collection in C#?

In ConcurrentStack, the Elements which is added last will be the element to be removed first. That means we are allowed to remove elements from the top of the ConcurrentStack. The ConcurrentStack Collection Class in C# provides the following methods to remove elements.

1. **TryPop(out T result):** This method attempts to pop and return the object at the top of the ConcurrentStack. Here, the output parameter result will contain the object removed if the operation was successful. If no object was available to be removed, the value is unspecified. It returns true if an element was removed and returned from the top of the ConcurrentStack successfully; otherwise, false.
2. **TryPopRange(T[] items):** This method attempts to pop and return multiple objects from the top of the ConcurrentStack atomically. The parameter items specify the Array to which objects popped from the top of the ConcurrentStack will be added. It returns the number of objects successfully popped from the top of the ConcurrentStack and inserted into items.
3. **TryPopRange(T[] items, int startIndex, int count):** This method attempts to pop and return multiple objects from the top of the ConcurrentStack atomically. Here, the parameter items specify the Array to which objects popped from the top of the ConcurrentStack will be added. The parameter startIndex specifies the zero-based offset in items at which to begin inserting elements from the top of the System.Collections.Concurrent.ConcurrentStack. And the parameter count specifies the number of

elements to be popped from the top of the ConcurrentStack and inserted into items. It returns the number of objects successfully popped from the top of the stack and inserted into items.

Let us see an example to understand the TryPop and TryPopRange methods of the ConcurrentStack<T> Collection Class in C#. Please have a look at the following example which shows the use of the TryPop and TryPopRange methods method.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            //Creating concurrentStack to store string values
            ConcurrentStack<string> concurrentStack = new ConcurrentStack<string>();
            //Adding Element using Push Method of ConcurrentStack Class
            concurrentStack.Push("India");
            concurrentStack.Push("USA");
            concurrentStack.Push("UK");
            concurrentStack.Push("Canada");
            concurrentStack.Push("Japan");
            concurrentStack.Push("Brazil");

            Console.WriteLine("All ConcurrentStack Elements:");
            foreach (var item in concurrentStack)
            {
                Console.WriteLine(item);
            }
            //Removing the top Element using TryPop Method
            bool IsRemoved = concurrentStack.TryPop(out string Result);
            Console.WriteLine($"\\nTryPop Return : {IsRemoved}");
            Console.WriteLine($"TryPop Result Value : {Result}");
            Console.WriteLine("\\nConcurrentStack Elements After TryPop Method");
            foreach (var item in concurrentStack)
            {
                Console.WriteLine(item);
            }

            //Creating a string array
            string[] countriesToRemove = { "UK", "NZ", "Brazil" };
            int NoOfCuntriesRemoved = concurrentStack.TryPopRange(countriesToRemove);
            Console.WriteLine($"\\nTryPopRange Return : {NoOfCuntriesRemoved}");
            Console.WriteLine("Elements Removed By TryPopRange Method");
            foreach (var item in countriesToRemove)
            {
                Console.WriteLine(item);
            }

            Console.WriteLine("\\nConcurrentStack Elements after TryPopRange Method");
        }
    }
}
```

```

        foreach (var item in concurrentStack)
        {
            Console.WriteLine(item);
        }
        Console.ReadKey();
    }
}

```

Output:

```

All ConcurrentStack Elements:
Brazil
Japan
Canada
UK
USA
India

TryPop Return : True
TryPop Result Value : Brazil

ConcurrentStack Elements After TryPop Method
Japan
Canada
UK
USA
India

TryPopRange Return : 3
Elements Removed By TryPopRange Method
Japan
Canada
UK

ConcurrentStack Elements after TryPopRange Method
USA
India

```

How to Get the Top Element from the ConcurrentStack in C#?

The `ConcurrentStack<T>` Collection Class in C# provides the following two methods to get the top element from the `ConcurrentStack` collection.

1. **TryPop(out T result):** This method attempts to pop and return the object at the top of the `ConcurrentStack`. Here, the output parameter `result` will contain the object removed if the operation was successful. If no object was available to be removed, the value is unspecified. It returns `true` if an element was removed and returned from the top of the `ConcurrentStack` successfully; otherwise, `false`.

2. **TryPeek(out T result):** This method attempts to return an object from the top of the ConcurrentStack without removing it. Here, the parameter result contains an object from the top of the ConcurrentStack or an unspecified value if the operation failed. It returns true if an object was returned successfully; otherwise, false.

For a better understanding, please have a look at the below example which shows how to get the top element from the ConcurrentStack using the TryPop(out T result) and TryPeek(out T result) methods of ConcurrentStack<T> Collection Class in C#.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            //Creating concurrentStack to store string values
            ConcurrentStack<string> concurrentStack = new ConcurrentStack<string>();
            //Adding Element using Push Method of ConcurrentStack Class
            concurrentStack.Push("India");
            concurrentStack.Push("USA");
            concurrentStack.Push("UK");
            concurrentStack.Push("Canada");
            concurrentStack.Push("Japan");
            //Accessing all the Elements of ConcurrentStack using For Each Loop
            Console.WriteLine($"ConcurrentStack Elements Count:
{concurrentStack.Count}");
            foreach (var item in concurrentStack)
            {
                Console.WriteLine(item);
            }
            // Removing and Returning the Top Element from ConcurrentStack using
method
            bool IsRemoved = concurrentStack.TryPop(out string Result1);
            Console.WriteLine($"\\nTryPop Return : {IsRemoved}");
            Console.WriteLine($"TryPop Result Value : {Result1}");
            //Printing Elements After Removing the Top Element
            Console.WriteLine($"\\nConcurrentStack Elements After TryPop: Count
{concurrentStack.Count}");
            foreach (var element in concurrentStack)
            {
                Console.WriteLine($"{element} ");
            }
            //Returning the Top Element from ConcurrentStack using TryPeek method
            bool IsPeeked = concurrentStack.TryPeek(out string Result2);
            Console.WriteLine($"\\nTryPeek Return : {IsPeeked}");
            Console.WriteLine($"TryPeek Result Value : {Result2}");
            //Printing Elements After TryPeek the Top Element
            Console.WriteLine($"\\nConcurrentStack Elements After TryPeek: Count
{concurrentStack.Count}");
        }
    }
}
```

```

        foreach (var element in concurrentStack)
        {
            Console.WriteLine($"{element} ");
        }

        Console.ReadKey();
    }
}

```

Output:

```

ConcurrentStack Elements Count: 5
Japan
Canada
UK
USA
India

TryPop Return : True
TryPop Result Value : Japan

ConcurrentStack Elements After TryPop: Count 4
Canada
UK
USA
India

TryPeek Return : True
TryPeek Result Value : Canada

ConcurrentStack Elements After TryPeek: Count 4
Canada
UK
USA
India

```

How to Copy a ConcurrentStack Collection to an Existing Array in C#?

In order to copy a ConcurrentStack Collection to an Existing Array in C#, we need to use the following CopyTo method of the ConcurrentStack Collection Class.

1. **CopyTo(T[] array, int index):** This method is used to copy the ConcurrentStack Elements to an existing one-dimensional Array, starting at the specified array index. Here, the parameter array specifies the one-dimensional array that is the destination of the elements copied from the ConcurrentStack. The Array must have zero-based indexing. The index parameter specifies the zero-based index in the array at which copying begins.

This method works on one-dimensional arrays and does not change the state of the ConcurrentStack. The elements are ordered in the array in the same way as the order of the elements from the beginning of the ConcurrentStack to the end. Let us see an example for a better understanding of the CopyTo(T[] array, int index) method of the ConcurrentStack<T> Collection Class in C#.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            //Creating concurrentStack to store string values
            ConcurrentStack<string> concurrentStack = new ConcurrentStack<string>();
            //Adding Element using Push Method of ConcurrentStack Class
            concurrentStack.Push("India");
            concurrentStack.Push("USA");
            concurrentStack.Push("UK");
            concurrentStack.Push("Canada");
            //Accessing all the Elements of ConcurrentStack using For Each Loop
            Console.WriteLine($"ConcurrentStack Elements");
            foreach (var item in concurrentStack)
            {
                Console.WriteLine(item);
            }
            //Copying the concurrentStack to an array
            string[] concurrentStackCopy = new string[5];
            concurrentStack.CopyTo(concurrentStackCopy, 0);
            Console.WriteLine("\nConcurrentStack Copy Array Elements:");
            foreach (var item in concurrentStackCopy)
            {
                Console.WriteLine(item);
            }

            Console.ReadKey();
        }
    }
}
```

Output:

```

ConcurrentStack Elements
Canada
UK
USA
India

ConcurrentStack Copy Array Elements:
Canada
UK
USA
India

```

ConcurrentStack<T> Collection Class with Complex Types in C#

As of now, we have used the ConcurrentStack Collection class with Primitive Data Types such as int, double, etc. Now, let us see how to use the ConcurrentStack Collection with complex types such as Employee, Student, Customer, Product, etc. For a better understanding, please have a look at the below example where we use the ConcurrentStack<T> Collection with the user-defined Student type.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentStackDemo
{
    class Program
    {
        static void Main()
        {
            //Creating concurrentStack to store string values
            ConcurrentStack<Student> concurrentStack = new ConcurrentStack<Student>();

            //Adding Elements to ConcurrentStack using Push Method
            concurrentStack.Push(new Student() { ID = 101, Name = "Anurag", Branch = "CSE" });
            concurrentStack.Push(new Student() { ID = 102, Name = "Mohanty", Branch = "CSE" });
            concurrentStack.Push(new Student() { ID = 103, Name = "Sambit", Branch = "ETC" });

            //Accessing all the Elements of ConcurrentStack using For Each Loop
            Console.WriteLine($"ConcurrentStack Elements");
            foreach (var item in concurrentStack)
            {
                Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch: {item.Branch}");
            }
            Console.ReadKey();
        }
    }
    public class Student

```

```

{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

All Concurrent Queue Elements
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC

```

Difference between Stack<T> and ConcurrentStack<T> in C#:

Stack<T>:

1. It is not thread-safe
2. It has the Pop method to remove the last inserted item from the collection.
3. Stack<T> can add a single item at a time using the Push method.
4. We can remove only a single item at a time using the Pop method.
5. In Stack<T>, we can remove items using the Pop method

ConcurrentStack<T>:

1. It is Thread-safe
2. ConcurrentStack has the TryPop method to remove the last inserted item from the collection.
3. ConcurrentStack<T> Can add multiple items at a time.
4. We can remove multiple items at a time using the TryPopRange method.
5. We can add items using the Push and PushRange method.
6. In ConcurrentStack<T>, we can remove items using TryPop or TryPopRange method.

In the next article, I am going to discuss the [ConcurrentBag<T> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **ConcurrentStack<T> Collection Class in C#** with Examples. I hope this ConcurrentStack Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.